Hybrid Neighborhood Evaluation in Tabu Search and Simulated Annealing for the Permutation Flow Shop Problem

Remigiusz Wojewódzki Wojciech Bożejko

remigiusz.wojewodzki@gmail.com wojciech.bozejko@pwr.edu.pl

Wrocław University
of Science and Technology

ICAISC 2026

Outline

Introduction

- **Permutation Flow Shop Problem (PFSP):** n jobs in the same order on m machines; minimize the makespan $C_{\max}$. NP-hard for $m \geq 3$.
- Neighborhood design is as important as the metaheuristic — it controls which moves are reachable per iteration.
- This paper compares three neighborhoods inside two classical frameworks (Tabu Search and Simulated Annealing):
  - **Adjacent swap** — baseline, single swap per iteration.
  - **Composite Disjoint-Adjacent** (nicknamed **Fibonacci**) — DP-selected set of non-overlapping adjacent swaps applied at once.
  - **Dynasearch** (recursive) — structured sequence of non-conflicting moves.
- Goal: understand the trade-off between exploration breadth, per-iteration cost, and convergence stability.

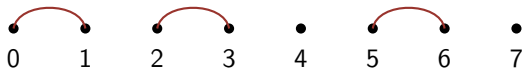
Composite Disjoint-Adjacent (a.k.a. *Fibonacci*)

Definition. Enumerate all single adjacent swaps $\pi^{(i)} = \mathrm{swap}(i, i{+}1)$ in π, with deltas

$$\Delta_i \;=\; C_{\max}\big(\pi^{(i)}\big) \;-\; C_{\max}(\pi), \qquad i = 1, \ldots, n{-}1.$$

A dynamic program selects a subset $S \subseteq \{1, \ldots, n{-}1\}$ of **non-overlapping** indices ($i \in S \Rightarrow i{+}1 \notin S$) minimizing $\sum_{i \in S} \Delta_i$. All selected swaps are then applied simultaneously.

Example — admissible composite move (swaps at 0, 2, 5):



Name origin. The number of valid subsets equals the Fibonacci number F_{n+1} — hence the nickname.

Algorithm & Complexity

Per iteration:

1. Recompute $C_{\max}(\pi^{(i)})$ for each $i = 1, \ldots, n-1$ — each $O(mn)$.
2. One-pass DP picks the optimal non-overlapping subset:
   $\mathrm{dp}[i] = \min(\mathrm{dp}[i+1],\ \Delta_i + \mathrm{dp}[i+2])$.
3. Apply selected swaps; if DP picks nothing, fall back to the least positive Δ_i.

Cost comparison (per iteration) — *Fibonacci* explores an exponentially-sized neighborhood (F_{n+1} valid subsets) at only quadratic cost:

Neighborhood	Candidates	Total per iter
Adjacent	$n-1$	$O(mn)$
Fibonacci	$n-1$ swaps + DP	$O(mn^2)$
Dynasearch (rec.)	$O(n^2)$ segments	$O(mn^3)$

Tabu Search & Simulated Annealing

Tabu Search (TS)

- Tabu list keyed by primitive moves (adjacent indices) or composite signatures (Fibonacci, Dynasearch).
- Tabu tenure $\tau_{\text{tabu}} = 10$.
- Aspiration: accept a tabu move if it improves the global best.
- At each step: pick the best admissible neighbor.

Simulated Annealing (SA)

- Multiplicative cooling $T \leftarrow \alpha T$ with floor T_{floor}.
- **Time-based reheating**: if no improvement for $\tau_{\text{stagn}} = 500$ ms and $T < T_0$, set $T \leftarrow \min(T_0, r \cdot T)$ $(r = 1.5)$.
- Standard Metropolis acceptance.

Same neighborhoods plugged into both — so any performance gap is attributable to the neighborhood, not the framework.

Experimental Setup

- **Platform:** MacBook Pro M4 Pro, 24 GB RAM, macOS 26.0.1, Python 3.11.
- **Benchmarks:** Taillard instances, $n \in \{20, 50, 100, 200, 500\}$, $m \in \{5, 10, 20\}$ — 12 instance classes.
- **Time limits:** $tl \in \{100, 500, 1000, 2000, 5000, 10000\}$ ms.
- **SA parameters:** $T_0 = 1000$, $T_{\text{final}} = 1$, $\alpha = 0.95$, reheat factor $r = 1.5$, stagnation 500 ms.
- **TS parameter:** $\tau_{\text{tabu}} = 10$.
- **Metric:** PRD — mean relative percentage deviation to best known $C^*_{\max}$:

$$\text{PRD} = \frac{C^{\text{obtained}}_{\max} - C^*_{\max}}{C^*_{\max}} \times 100\%.$$

Results — PRD at $tl = 5{,}000$ ms (selected)

instance	n	m	fib_{TS}	fib_{SA}	dyn_{TS}	dyn_{SA}	adj_{TS}	adj_{SA}
tai20_5	20	5	17.39	17.39	6.18	6.18	**4.17**	6.24
tai50_10	50	10	20.23	20.23	**6.95**	6.95	17.10	9.72
tai100_10	100	10	17.19	17.19	**6.57**	6.48	16.52	8.27
tai200_10	200	10	**12.71**	12.71	13.20	13.20	13.19	8.53
tai200_20	200	20	**18.60**	18.60	20.71	20.71	19.83	20.20
tai500_20	500	20	13.91	13.90	14.51	14.51	**15.34**	15.82

- Small n ($n \leq 100$): Dynasearch wins.
- Crossover at $n = 200$: *Fibonacci* **overtakes both** Adjacent and Dynasearch.
- At $n = 500$: Dynasearch and Adjacent fall behind; *Fibonacci* stays competitive.

Discussion

- **Early-budget gain.** The composite move bundles several non-overlapping improving swaps into one iteration — a steeper initial drop in $C_{\max}$ than Adjacent.
- **Plateau risk.** Once all single $\Delta_i \geq 0$, the composite degenerates to a single swap and stagnates. **SA reheating** partially mitigates this by re-injecting diversification.
- **Dynasearch cost.** A single Dynasearch iteration at $n = 500$ can exceed tl by $\sim 200\times$ — so it never completes one iteration within short budgets.
- **Practical implication.** *Fibonacci* is the right default for medium-to-large n under tight time limits; Adjacent for very small n; Dynasearch only when ample budget is available.

Conclusion and Future Work

Summary:

- Three neighborhood structures (Adjacent, *Fibonacci*, Dynasearch) compared inside TS and SA under identical time budgets on Taillard.
- Composite (*Fibonacci*) neighborhood accelerates early descent and dominates from $n = 200$ onward.
- Dynasearch reaches deeper optima but is impractical for $n \geq 200$ within short time budgets.
- SA's time-based reheating helps escape plateaus that the composite move induces.

Future work:

- Incremental delta evaluation to cut per-iteration cost of composite moves.
- Adaptive switching between neighborhood structures during the run.
- Probabilistic variant sampling and comparison with state-of-the-art FSP solvers (e.g. iterated local search, VND).